

Case Study

A small library wants to maintain book and member details digitally. The librarian should be able to add books, view books, add members, and view members using a Web API.

You need to create a Web API using **ASP.NET Core** and **EF Core** with proper layered architecture.

Required Layers

Create the project using the following layers:

LibraryManagement.API

LibraryManagement.Models

LibraryManagement.Data

LibraryManagement.Repository

LibraryManagement.Service

Database Tables

1. Book

Column	Type	Description
BookId	int	Primary Key
Title	string	Book title
Author	string	Author name
ISBN	string	Book ISBN number
PublishedYear	int	Published year
AvailableCopies	int	Number of copies available

2. Member

Column	Type	Description
MemberId	int	Primary Key

FullName	string	Member name
Email	string	Member email
PhoneNumber	string	Contact number
MembershipDate	DateTime	Date of joining

Required APIs

Book APIs

1. Add Book

POST /api/books

Sample Request:

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "isbn": "9780132350884",
  "publishedYear": 2008,
  "availableCopies": 5
}
```

Expected Response:

```
{
  "message": "Book added successfully"
}
```

2. Get All Books

GET /api/books

Expected Response:

```
[
  {
    "bookId": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "isbn": "9780132350884",
```

```
    "publishedYear": 2008,  
    "availableCopies": 5  
  }  
]
```

3. Get Book By Id

GET /api/books/1

Member APIs

4. Add Member

POST /api/members

Sample Request:

```
{  
  "fullName": "Rahul Sharma",  
  "email": "rahul@gmail.com",  
  "phoneNumber": "9876543210",  
  "membershipDate": "2026-05-20"  
}
```

Expected Response:

```
{  
  "message": "Member added successfully"  
}
```

5. Get All Members

GET /api/members

6. Get Member By Id

GET /api/members/1

Required Implementation

Students should implement:

Models

Book.cs
Member.cs

DbContext

LibraryDbContext.cs

Repository Layer

IBookRepository.cs
BookRepository.cs
IMemberRepository.cs
MemberRepository.cs

Service Layer

IBookService.cs
BookService.cs
IMemberService.cs
MemberService.cs

API Layer

BooksController.cs
MembersController.cs

Business Rules

1. Book title should not be empty.
2. Author name should not be empty.
3. Available copies should be greater than or equal to 0.
4. Member full name should not be empty.
5. Email should not be empty.
6. Phone number should not be empty.
7. If a book or member is not found by ID, return 404 Not Found.

Student Tasks

1. Create an ASP.NET Core Web API project.
2. Install EF Core packages.
3. Create models for Book and Member.
4. Create LibraryDbContext.

5. Configure SQL Server connection string.
6. Create repository interfaces and classes.
7. Create service interfaces and classes.
8. Create API controllers.
9. Register dependencies in Program.cs.
10. Create migration and update database.
11. Test all APIs using Swagger or Postman.

Expected Folder Structure

LibraryManagement.API

```
|
|—— Controllers
|   |—— BooksController.cs
|   |—— MembersController.cs
|
```

LibraryManagement.Models

```
|
|—— Book.cs
|—— Member.cs
|
```

LibraryManagement.Data

```
|
|—— LibraryDbContext.cs
|
```

LibraryManagement.Repository

```
|
|—— IBookRepository.cs
|—— BookRepository.cs
|—— IMemberRepository.cs
|—— MemberRepository.cs
|
```

LibraryManagement.Service

```
|
|—— IBookService.cs
|—— BookService.cs
|—— IMemberService.cs
|—— MemberService.cs
```

Submission Requirements

Students must submit:

1. Complete source code.
2. Database migration files.
3. Screenshot of Swagger showing all APIs.
4. Screenshot of successful POST request.
5. Screenshot of successful GET request.
6. SQL Server database screenshot showing Books and Members tables.

Bonus Task

Add one more API:

GET /api/books/search?title=clean

This API should return books matching the given title.